

SMART SERVICE: AN AI-ASSISTED MOBILE APPLICATION FOR CONNECTING CUSTOMERS WITH SERVICE PROVIDERS

Senior Project

by

Mira Youssef, 72030174

Hussein Mansour, 42030321

Submitted to the School of Engineering of the

Lebanese International University

Nabatieh, Lebanon

in partial fulfillment of the requirements for the degree of

**BACHELOR OF SCIENCE IN COMPUTER AND
COMMUNICATION
ENGINEERING**

Fall 2026-2027

Approved by:

Supervisor

Dr. Hussein Abdallah

Committee Member

DEDICATION

Mira Youssef dedicates this work to her family, whose patience, encouragement, and confidence made every difficult stage achievable, and to every learner who turns persistence into meaningful engineering.

Mira Youssef

Hussein Mansour dedicates this project to his family and mentors for their constant support, and to all service professionals whose daily work inspired the purpose and practical direction of Smart Service.

Hussein Mansour

ACKNOWLEDGMENT

We express our sincere gratitude to our supervisor, Dr. Hussein Abdallah, for his guidance, constructive observations, and continued encouragement throughout the development of this graduation project. We also thank the faculty members of the School of Engineering at the Lebanese International University for the technical foundation and academic discipline that supported this work. Finally, we acknowledge our families and colleagues for their patience, feedback, and motivation during the design, implementation, testing, and documentation of Smart Service.

ABSTRACT

Smart Service is an AI-assisted mobile application that connects customers who require household maintenance with nearby service providers in three initial categories: electrical work, plumbing, and air-conditioning. The system consists of a JavaScript React Native application developed with Expo, a Node.js and Express REST API, and a MongoDB Atlas database. Customers can register, authenticate, capture their current GPS location, describe a problem, attach an optional image, and discover available providers who offer the required service within a configurable radius. Results are ordered by straight-line distance and show provider ratings and an approximate arrival time. The customer retains the decision to select a provider, while the provider may accept or reject the offer and advance an accepted request through controlled lifecycle states. Completed services can be rated, and both participants can remove eligible historical requests from their own lists through soft deletion. Two explainable rule-based analysis modules classify urgency and produce an initial price range and duration estimate from the service type and bilingual English/Arabic descriptions. These modules do not select a provider, charge a customer, or replace professional inspection. The implemented prototype demonstrates role-based authentication, geospatial filtering, state validation, local image handling, feedback aggregation, and transparent decision support in a unified mobile workflow.

TABLE OF CONTENTS

DEDICATION	2
ACKNOWLEDGMENT	3
ABSTRACT	4
TABLE OF CONTENTS	5
LIST OF FIGURES	7
LIST OF TABLES	8
LIST OF SYMBOLS AND ABBREVIATIONS	9
CHAPTER 1 INTRODUCTION	10
1.1 Background.....	10
1.2 Problem Statement.....	11
1.3 General Overview of the Project	12
1.4 Thesis Outline.....	17
CHAPTER 2 SURVEY OF EXISTING METHODS AND SIMILAR SYSTEMS	19
2.1 Introduction	19
2.2 Similar System 1: Taskrabbat	19
2.3 Similar System 2: Angi	20
2.4 Similar System 3: Urban Company	20
2.5 Methods/Systems Comparison	21
2.6 Conclusion and Motivation	22
CHAPTER 3 SYSTEM DESIGN	23

3.1 Introduction	23
3.2 Requirements and Specification Analysis	23
3.2.1 Functional Requirements.....	23
3.2.2 Use Case Diagrams.....	25
3.3 System Architecture	25
3.4 Class Diagrams	26
3.5 Sequence Diagrams	27
3.6 Activity Diagrams	28
3.7 Entity-Relationship Diagrams	29
3.8 Non-Technical Aspects.....	30
3.8.1 Financial Viability	30
3.8.2 Stakeholders	31
3.8.3 Scope	32
3.8.4 Risks	32
3.8.5 Schedule and Milestones	32
3.8.6 Ethical and Social Considerations	33
3.8.7 Environmental and Sustainability Considerations.....	34
3.8.8 Relevant Standards	34
3.9 Conclusion.....	35
REFERENCES	36

LIST OF FIGURES

<u>Figure 1-1. Guest access, registration, and login interfaces.</u>	13
<u>Figure 1-2. Customer home, GPS-enabled request creation, and request list.</u>	14
<u>Figure 1-3. Nearby available-provider discovery with distance and approximate arrival.</u>	15
<u>Figure 1-4. Customer rating and provider review interfaces.</u>	17
<u>Figure 1-5. Provider profile configuration and incoming-request interface.</u>	16
<u>Figure 3-1. Customer and provider use-case diagram.</u>	25
<u>Figure 3-2. Smart Service client-server system architecture.</u>	26
<u>Figure 3-3. Main class and domain-model diagram.</u>	27
<u>Figure 3-4. Service-request interaction sequence diagram.</u>	28
<u>Figure 3-5. Service-request lifecycle activity diagram.</u>	29
<u>Figure 3-6. MongoDB entity-relationship diagram.</u>	30
<u>Figure 3-7. Prototype cost-benefit overview.</u>	31
<u>Figure 3-8. Project schedule and milestones.</u>	33

LIST OF TABLES

<u>Table 2-1. Comparison based on graphical interface and usability.</u>	21
<u>Table 2-2. Comparison based on content and functionality.</u>	21
<u>Table 2-3. Comparison based on core features.</u>	22
<u>Table 3-1. Customer functional requirements.</u>	23
<u>Table 3-2. Provider functional requirements.</u>	24
<u>Table 3-3. Server functional requirements.</u>	24
<u>Table 3-4. Principal stakeholders.</u>	31
<u>Table 3-5. Prototype financial assumptions.</u>	30
<u>Table 3-6. Principal project risks and mitigations.</u>	32
<u>Table 3-7. Schedule and milestone summary.</u>	32
<u>Table 3-8. Relevant standards and guidance.</u>	34

LIST OF SYMBOLS AND ABBREVIATIONS

- AI** — Artificial Intelligence
- API** — Application Programming Interface
- DB** — Database
- ERD** — Entity–Relationship Diagram
- GPS** — Global Positioning System
- HTTP** — Hypertext Transfer Protocol
- JWT** — JSON Web Token
- LIU** — Lebanese International University
- REST** — Representational State Transfer
- UI** — User Interface
- UML** — Unified Modeling Language
- URL** — Uniform Resource Locator

CHAPTER 1 INTRODUCTION

1.1 Background

Mobile service marketplaces reduce the effort required to identify, compare, and contact people who provide work at a customer's location. Instead of depending only on informal recommendations, a customer can describe a need through a structured interface and review information about matching providers. Established marketplaces demonstrate that provider skills, availability, price information, location, and reviews are central to service discovery. Taskrabbit, for example, officially describes a flow in which customers choose a category, enter an address, compare profiles by skills, hourly rate, and reviews, and then select a Tasker [1]. Angi similarly supports researching local professionals, booking services, requesting quotes, and consulting cost information [2].

A cross-platform mobile architecture is appropriate for such a system because the same interface must support customers and providers on multiple devices. React Native allows application interfaces to be expressed through React components while targeting native platforms [3]. Expo adds managed access to device capabilities; its Location module supports foreground permission requests and current-position acquisition [4]. Smart Service uses these capabilities for one-time coordinate capture rather than continuous tracking.

Location-aware discovery requires a consistent geographic data representation. GeoJSON represents a point using longitude followed by latitude, and RFC 7946 defines the interoperable format [5]. MongoDB supports GeoJSON points, spherical distance calculations, and 2dsphere indexes; its \$geoNear stage can return an ordered stream based on proximity [6]. These facilities allow Smart Service to filter by role, selected service,

current availability, valid location, rejection history, and a configurable radius before sorting candidates by straight-line distance.

Security and traceability are also necessary because request data belongs to identifiable participants. Smart Service hashes passwords using bcrypt and issues seven-day JSON Web Tokens containing only the user identifier and role. RFC 7519 defines JWT as a compact claims representation [7], while OWASP advises that passwords should be stored with deliberately slow password-hashing algorithms rather than as plaintext [8]. Role middleware separates customer-only and provider-only operations, and lifecycle updates use expected-state checks to prevent skipping or repeating transitions.

The term AI-assisted in this project refers to explainable, deterministic analysis rather than generative AI. English and Arabic problem descriptions are normalized and evaluated against configured indicators. One service assigns a priority level and reason; another assigns a bounded initial price range, duration, currency, and explanation. Urgent wording by itself does not raise the price. The provider must inspect the problem and confirm the final price, and the customer always chooses the provider.

1.2 Problem Statement

Customers who need electricians, plumbers, or air-conditioning technicians may face fragmented information about who is currently available, what services each person offers, how far away the provider is, and whether earlier customers were satisfied. A simple directory does not represent the operational state of a request, while direct messaging alone does not enforce a reliable sequence from an offer to completion. Providers likewise need a concise mechanism to state their offered services and availability, receive relevant work, and avoid being shown for categories they do not perform.

The project problem is therefore to design a role-aware mobile system that creates a trustworthy, inspectable connection between a customer request and an eligible nearby provider. The system must preserve customer choice, disclose only safe information before acceptance, prevent unauthorized status changes, and keep both participants informed despite the absence of real-time sockets. It must also handle device location permission, inconsistent connectivity, duplicate actions, rejected offers, historical cleanup, and optional images without exposing passwords, tokens, database credentials, or exact provider coordinates in discovery results.

A secondary problem is that free-text descriptions vary in language and detail. A severe flood or electrical sparks should be visually distinguished from routine maintenance, but a software estimate must not pretend to replace professional judgment. Smart Service addresses this with transparent keyword rules, bounded outputs, human-readable reasons, and explicit warnings. The analysis prioritizes requests for provider presentation; it does not automatically assign a provider, authorize payment, or establish a final service price.

1.3 General Overview of the Project

Smart Service has two authenticated roles. A customer registers or logs in, selects electrician, plumber, or air-conditioning service, writes a meaningful description, optionally selects one JPEG, PNG, or WebP image, and captures a foreground GPS position. When the backend creates the request, it constructs the GeoJSON location, performs priority and service estimation, and stores the results. The customer's request list displays status, analysis, estimate, location presence, provider information, lifecycle progress, rating state, and eligible cleanup actions.

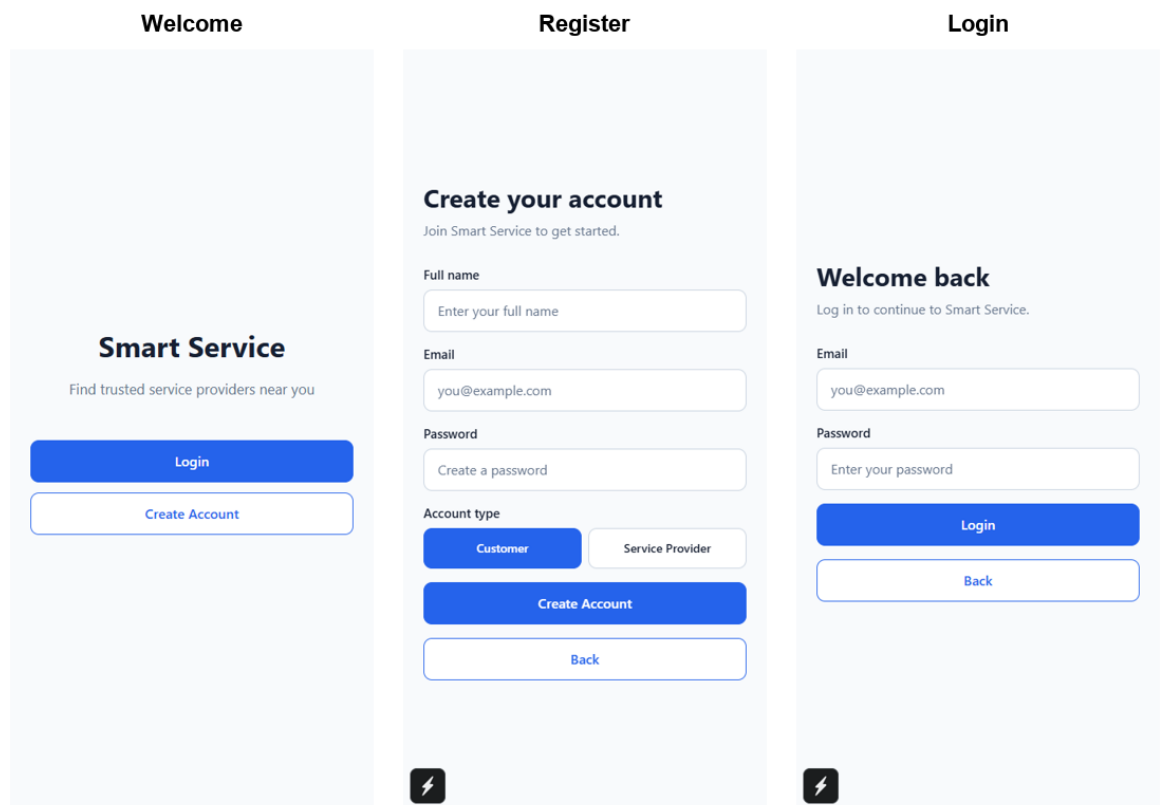


Figure 1-1. Guest access, registration, and login interfaces.

Figure 1-1 shows the genuine Expo Web guest workflow used by both account types. The navigation tree is replaced after authentication, so an authenticated user cannot return to Login through the Back action. Tokens use SecureStore on Android and iOS and sessionStorage only as a web-development fallback.

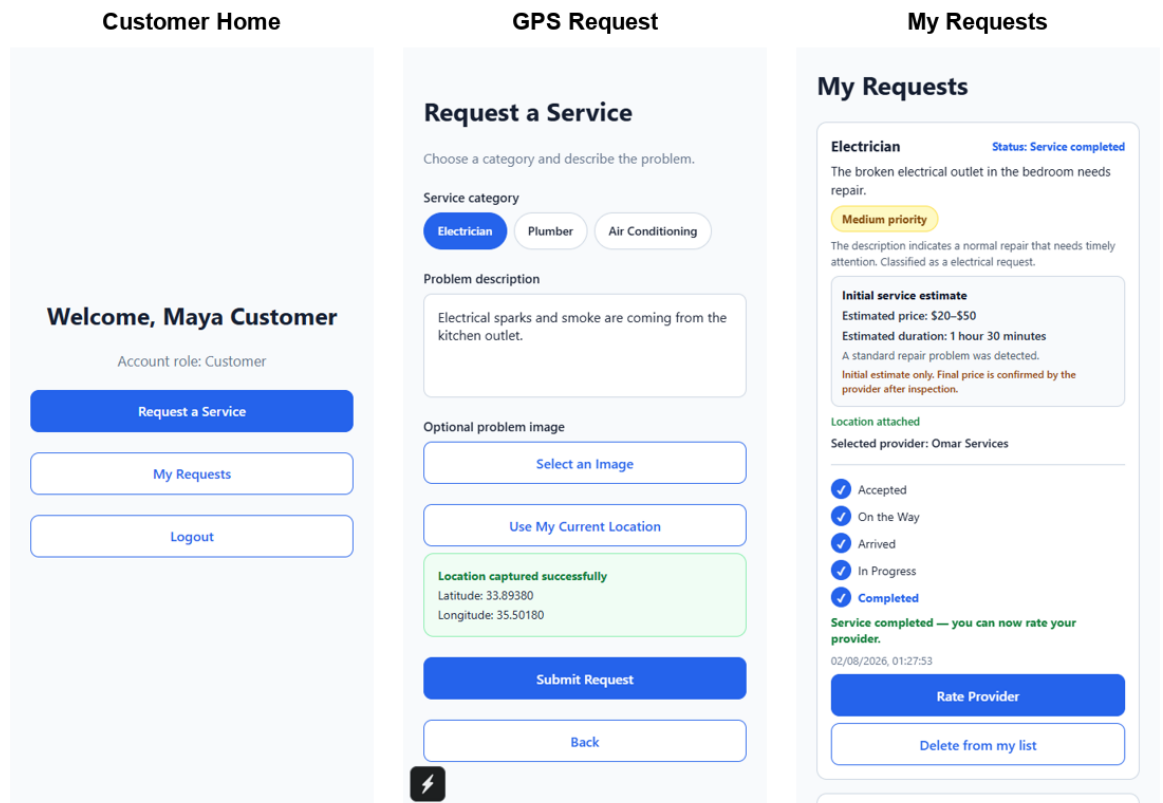


Figure 1-2. Customer home, GPS-enabled request creation, and request list.

As illustrated in Figure 1-2, the customer workflow combines service selection, optional image input, one-time GPS capture, and a request list. New requests require valid coordinates, but older database records with null locations remain valid. Background polling refreshes request information every five seconds while the list screen is focused and stops when the screen loses focus.

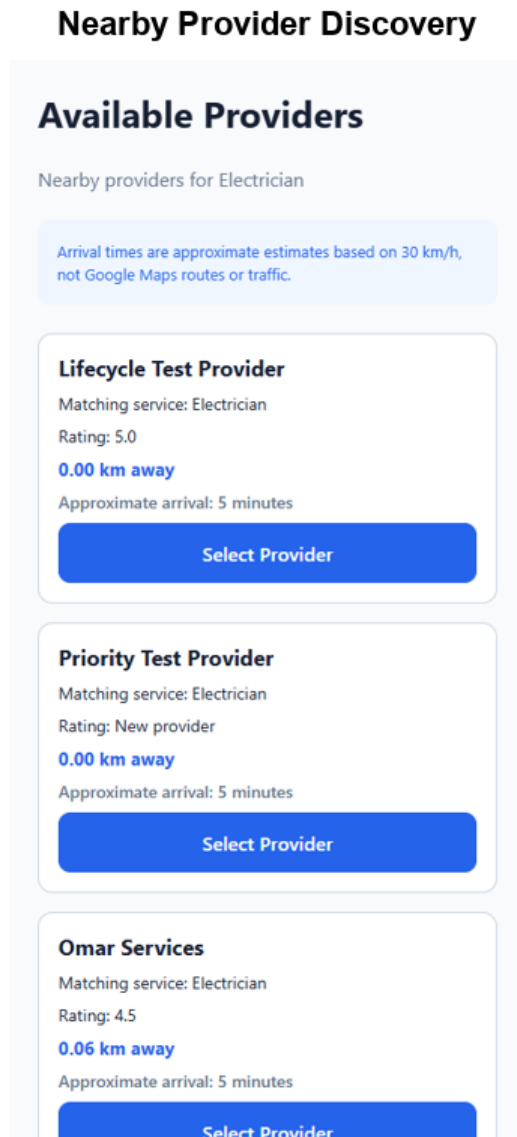


Figure 1-3. Nearby available-provider discovery with distance and approximate arrival.

The discovery result in [Figure 1-3](#) includes only providers with the provider role, available status, matching service, valid location, and no previous rejection for that request. Exact provider coordinates and email are withheld. Distance is calculated by MongoDB, and arrival time is a clearly labeled approximation based on 30 km/h with a five-minute minimum; it is not a route or traffic prediction.

Provider Profile

Welcome, Omar Services

Account role: Service Provider

Current saved status

Available

Customer rating

4.5 / 5

2 ratings

Your services

Select one or more services you provide.

Electrician

Plumber

Air Conditioning

Your current location

Update My Current Location

Location saved successfully

Latitude: 33.89420

Longitude: 35.50220

Last updated: 02/08/2026, 01:27:51

Availability

Available

Busy

Save

Incoming Requests

Incoming Requests

Active requests

Maya Customer

Service: Plumber

There is a small water leak under the kitchen sink.

Medium priority

No emergency or routine-maintenance keywords were found, so this request has normal priority.

Initial service estimate

Estimated price: \$20-\$55

Estimated duration: 1 hour 30 minutes

A standard repair problem was detected.

Initial estimate only. Final price is confirmed by the provider after inspection.

Distance: 0.06 km

Created: 02/08/2026, 01:27:52

Accept

Reject

Completed history

Maya Customer

Service: Electrician

The broken electrical outlet in the bedroom needs repair.

Medium priority

Completed

Figure 1-5. Provider profile configuration and incoming-request interface.

The provider profile and incoming-request views appear in [Figure 1-5](#). A provider must save at least one service and a location before becoming available. Offered requests may be accepted or rejected. Acceptance marks the provider busy; rejection preserves history and permits the customer to find another provider. Accepted work advances only through accepted, on the way, arrived, in progress, and completed. Completion can restore availability when service and location requirements remain satisfied.

Customer Rating

Rate Provider

How was your service with Omar Services?

★ ★ ★ ★ ★

Select a score

Optional comment

0/500

Submit Rating

Back

Provider Reviews

My Reviews

4.5
2 ratings

Maya Customer
★★★★★

Air Conditioning
Professional service, clear communication, and excellent work.
02/08/2026, 01:27:53

Back to Provider Home

Figure 1-4. Customer rating and provider review interfaces.

After completion, the owning customer may submit one score from one to five and an optional comment. The backend prevents duplicate ratings, records the rating reference, and recalculates the provider average to one decimal place. Figure 1-4 shows the rating and provider-review interfaces. Neither participant permanently deletes the shared request through list cleanup; separate hidden flags remove eligible rejected, completed, or cancelled records only from that participant’s view.

1.4 Thesis Outline

This preliminary report contains three completed chapters. Chapter 1 introduces the service-marketplace problem, project objectives, implemented scope, and principal technologies. Chapter 2 surveys Taskrabbit, Angi, and Urban Company using official information, then compares their documented functions with Smart Service. Chapter 3

specifies functional requirements, diagrams the implemented architecture and data model, explains the request workflow, and examines financial, stakeholder, scope, risk, schedule, ethical, environmental, and standards considerations. Implementation testing and final conclusions will be addressed in later report stages and are intentionally not presented as Chapters 4 and 5 here.

CHAPTER 2 SURVEY OF EXISTING METHODS AND SIMILAR SYSTEMS

2.1 Introduction

This chapter reviews three established home-service marketplaces to identify common interaction patterns and unresolved design opportunities. Only capabilities established by the reviewed official sources are treated as verified. A blank or “Not established” comparison entry means that the selected source did not provide sufficient evidence; it does not prove that the commercial system lacks the feature.

2.2 Similar System 1: Taskrabbt

Taskrabbt connects customers with independent Taskers across multiple service categories. Its official hiring guidance instructs customers to select a category, provide the task address, review local Tasker profiles, and compare skills, experience, hourly rates, and category-specific reviews before selecting a Tasker [1]. The platform supports scheduling and task details, and its public overview describes integrated chat, payment, tipping, and reviews [9].

Taskrabbt strongly demonstrates customer-directed provider selection and profile comparison. Taskers can define skills, location, availability slots, and a work-area map [10]. Pricing is provider-controlled, while the platform may offer market-based pricing guidance [11]. The reviewed materials did not establish explainable emergency-text prioritization comparable to Smart Service, nor did they establish a bilingual rule that produces a request-specific initial duration estimate from the problem description.

2.3 Similar System 2: Angi

Angi positions itself as a platform for finding, booking, and hiring skilled home professionals, researching costs, requesting quotes, and consulting homeowner reviews [2]. Search results present professional information, ratings, and profile links. Angi also reports transparent upfront pricing for services booked and paid through its platform and supplies a project cost center based on project data [2].

The Angi workflow covers a wider home-improvement planning context than the three-category Smart Service prototype. Recent official professional guidance describes location-and-service matching between homeowner projects and professional opportunities [12]. However, the reviewed public documentation does not establish the same explicit customer-controlled offer state, provider rejection history, deterministic request priority analysis, or five-state field-service lifecycle implemented by Smart Service.

2.4 Similar System 3: Urban Company

Urban Company is a technology-enabled home-services marketplace whose stated mission includes enabling service professionals to deliver at-home services [13]. Its public material emphasizes professional enablement, training, service quality, minimum rating expectations, and on-time fulfillment. These concerns are directly relevant to provider quality and customer trust.

Urban Company illustrates a managed-service approach in which platform processes influence professional preparation and quality assurance. The reviewed official sources establish attention to ratings and fulfillment, but do not establish the same explainable bilingual keyword prioritization, customer-selected nearby-provider list, or locally stored optional request image used in the current Smart Service implementation.

Smart Service remains a prototype and does not claim the scale, verification programs, support infrastructure, or commercial maturity of Urban Company.

2.5 Methods/Systems Comparison

Tables 2-1 through 2-3 compare documented characteristics. Smart Service entries are verified from the inspected source code and captured running interface. Commercial-system entries are deliberately conservative and are limited to the cited official materials.

Table 2-1. Comparison based on graphical interface and usability.

Criterion	Taskr abbit	Angi	Urban Company	Smart Service
Mobile-oriented service flow	Verified	Verified	Verified	Expo UI Verified
Provider profile/ratings visible	Verified	Verified	Quality/rating emphasis	Verified
Customer directly chooses provider	Verified	Available in directory/booking flows	Not established by reviewed source	Verified
Simple role-specific navigation	Separate client/Tasker apps	Home owner/pro experiences	Customer/professional experiences	Separate navigation trees
Accessible labels/status messages	Not assessed	Not assessed	Not assessed	Partially implemented; WCAG review remains

Table 2-2. Comparison based on content and functionality.

Criterion	Taskr abbit	Angi	Urban Company	Smart Service
Service discovery	Category/address search	Professional search and booking	At-home service marketplace	Three service categories
Location input/support	Task address and work area	Local professional matching	On-time at-home fulfillment	GPS + GeoJSON proximity
Price information	Hourly rate and pricing guidance	Upfront pricing/cost research	Service marketplace pricing	Initial rule-based range only
Reviews	Integrated reviews	Homeowner reviews	Rating s/quality thresholds	One rating per completed request
Provider workflow	Skills, work area, schedule	Professional opportunities/leads	Professional enablement	Services, location, availability, lifecycle

Table 2-3. Comparison based on core features.

Feature	Taskr	Angi	Urba	Smart
	abbi	n Company	Service	
GPS-distance ordered candidates	Not established	Local matching; ordering not established	Not established	Yes, MongoDB \$geoNear
Provider accept/reject offer	Scheduling confirmation documented	Mutual lead interest documented	Not established	Yes
Detailed request status lifecycle	Not established	Not established	Fulfillment emphasized	Yes, five post-acceptance states
Optional problem image	Not established	Not established	Not established	Yes, local Multir upload
Emergency priority explanation	Not established	Not established	Not established	Yes, English/Arabic rules
Problem-based price/duration estimate	Pricing guidance, not equivalent	Cost/ upfront pricing, not equivalent	Not established	Yes; preliminary only
Customer retains provider selection	Yes	Supported in relevant flows	Not established	Yes; AI never assigns

2.6 Conclusion and Motivation

The reviewed systems demonstrate the value of category-based search, location context, provider profiles, ratings, price information, and structured marketplace participation. Their scale and operational maturity exceed the current academic prototype. The comparison nevertheless identifies a coherent project motivation: combine transparent customer selection, provider-controlled acceptance, GPS-radius filtering, explicit lifecycle validation, bilingual emergency prioritization, and an explainable initial estimate in one inspectable workflow.

Smart Service is not intended to duplicate every commercial function. It deliberately excludes payment, chat, background-verification infrastructure, traffic routing, and automatic provider assignment. Its contribution is a focused engineering demonstration in which analysis remains explainable, authorization is role-specific, customer choice is preserved, and database state transitions can be tested directly.

CHAPTER 3 SYSTEM DESIGN

3.1 Introduction

This chapter translates the project goals into verified functional requirements and design models. The diagrams are derived from the inspected application rather than copied from the report template. They describe the deployed software boundaries, principal domain entities, API-mediated interactions, lifecycle states, and MongoDB relationships, followed by the non-technical considerations required by the template.

3.2 Requirements and Specification Analysis

Smart Service follows a role-based client-server model. Every protected API operation first verifies a Bearer JWT and then applies customer or provider authorization. Customer and provider screens share authentication state through AuthContext, while request-oriented screens reload on focus and poll every five seconds. The backend remains authoritative for ownership, coordinates, eligible providers, request state, rating uniqueness, and list visibility.

3.2.1 Functional Requirements

Table 3-1. Customer functional requirements.

D	Customer requirement	Implementation evidence
1	Register and authenticate	Register/Login screens; /api/auth routes; JWT context
2	Capture one current location	Expo Location foreground permission
3	Create categorized request	POST /api/requests; service enum and description validation
4	Attach at most one optional image	Expo Image Picker; Multer type/size controls
5	View nearby matching available providers	\$geoNear, 50 km default, service/status filters
6	Select one provider	Atomic select-provider endpoint; customer ownership

D	Customer requirement	Implementation evidence
7	Observe provider response/lifecycle	Five-second focused polling and progress timeline
8	View priority and initial estimate	Shared priority badge and estimate component
9	Rate completed request once	Unique Rating request reference and ownership checks
10	Hide eligible history from own list	Per-customer soft visibility flag

Table 3-2. Provider functional requirements.

D	Provider requirement	Implementation evidence
1	Register and authenticate as provider	Role enum, JWT, provider navigation
2	Select offered service types	Validated electrician/plumber/AC array
3	Save current location and availability	Provider GeoJSON endpoint and profile validation
4	Receive assigned offered/active requests	Provider-only request aggregation
5	Accept or reject offered request	Atomic response endpoint
6	Advance exact lifecycle order	Expected-current-status atomic update
7	View customer-safe request data	No pre-acceptance email/exact coordinates
8	View priority, estimate, and reviews	Shared analysis cards and ratings endpoint
9	Hide eligible history from own list	Per-provider soft visibility flag

Table 3-3. Server functional requirements.

D	Server requirement	Implementation evidence
1	Authentication and role authorization	JWT middleware and authorizeRoles
2	Persist users, requests, ratings	Mongoose schemas and MongoDB Atlas
3	Validate geographic coordinates	Server builds Point [longitude, latitude]
4	Filter and rank providers	Role/service/status/location/rejection filters
5	Protect request state transitions	Atomic conditions and conflict responses
6	Manage images safely	Unique names, MIME allowlist, 5 MB maximum
7	Analyze priority and service estimate	Centralized deterministic rule services
8	Aggregate ratings	Provider average rounded to one decimal place

3.2.2 Use Case Diagrams

Figure 3-1 separates customer and provider goals while showing that both roles depend on the same authenticated system boundary. Provider selection remains a customer use case; analysis produces supporting information but does not execute that decision.

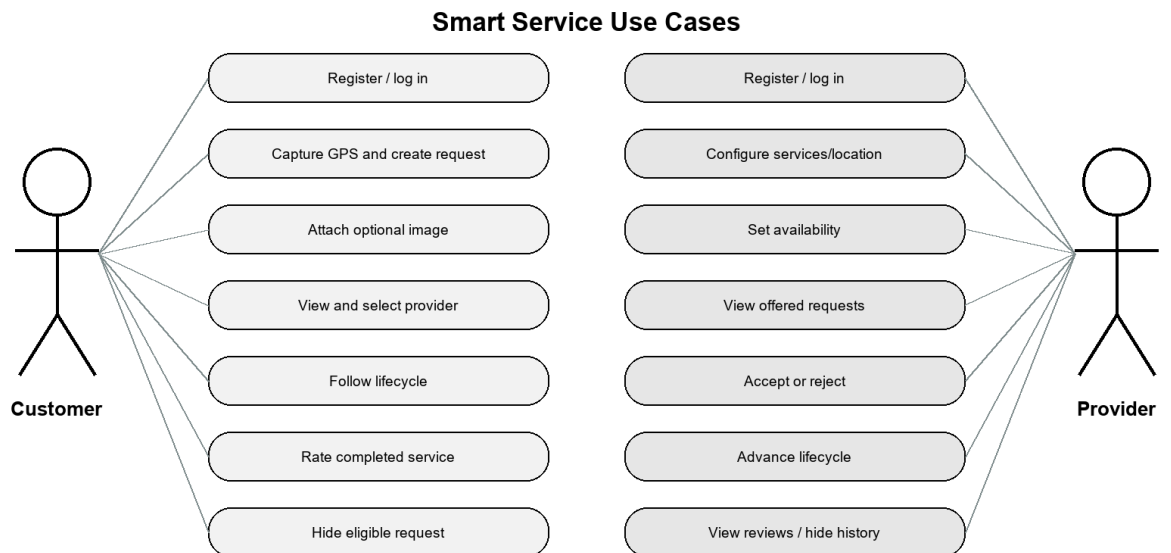


Figure 3-1. Customer and provider use-case diagram.

3.3 System Architecture

The architecture in Figure 3-2 has four principal runtime elements. The Expo client owns presentation, navigation, permission prompts, transient form state, secure token storage, and polling. The Express API owns validation, authorization, workflow rules, analysis, and response shaping. MongoDB Atlas stores structured entities and supports spherical proximity operations. The backend uploads directory stores the optional image file and is exposed through a static Express route. No database URI or JWT secret is included in the mobile environment.

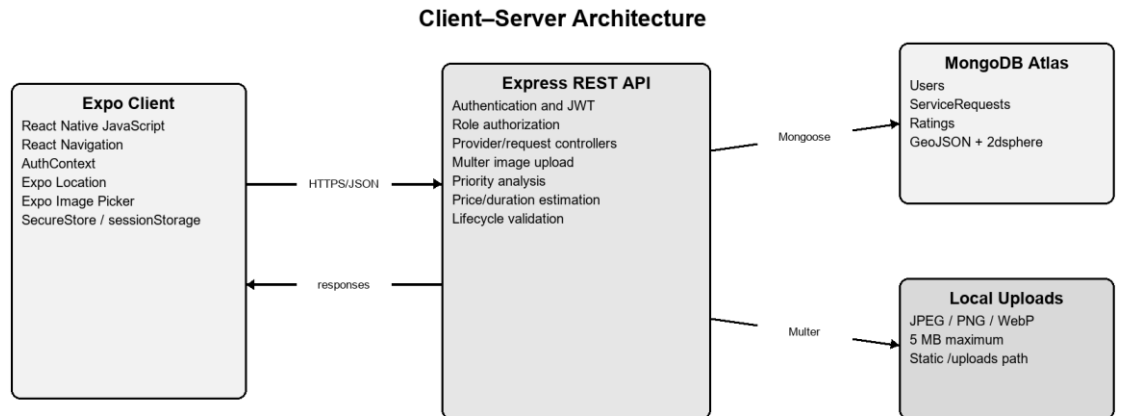


Figure 3-2. Smart Service client-server system architecture.

Communication uses JSON for ordinary requests and multipart/form-data when a service request includes an image. The mobile client sends only latitude and longitude; the server constructs the GeoJSON point. Protected requests carry the JWT in the Authorization header as a Bearer token. CORS is enabled for development. Production deployment would additionally require HTTPS, restrictive CORS origins, managed object storage, monitoring, backup policy, and secret management.

3.4 Class Diagrams

Because the implementation uses JavaScript modules and Mongoose rather than a classical object-oriented hierarchy, [Figure 3-3](#) is a domain-oriented class diagram. It shows stored model properties and the application services that operate on them. Password is excluded from normal queries. ServiceRequest contains both workflow state and immutable analysis outputs generated at creation time; old requests without estimates are displayed as unavailable rather than silently migrated.

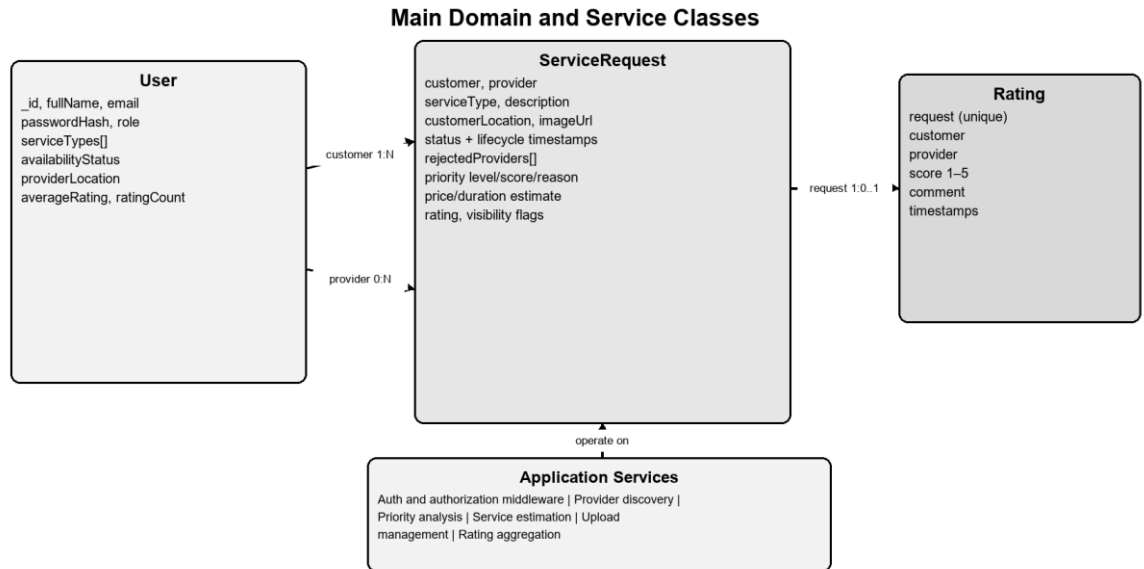


Figure 3-3. Main class and domain-model diagram.

3.5 Sequence Diagrams

Figure 3-4 models the primary request sequence. The customer client sends the validated request to Express, which computes priority and estimate, creates GeoJSON, and stores the document. Provider discovery invokes a spherical MongoDB aggregation. Selection creates an offer for one provider; the provider’s separate navigation then obtains that offer. Subsequent acceptance, rejection, lifecycle, and rating operations follow the same authenticated API boundary.

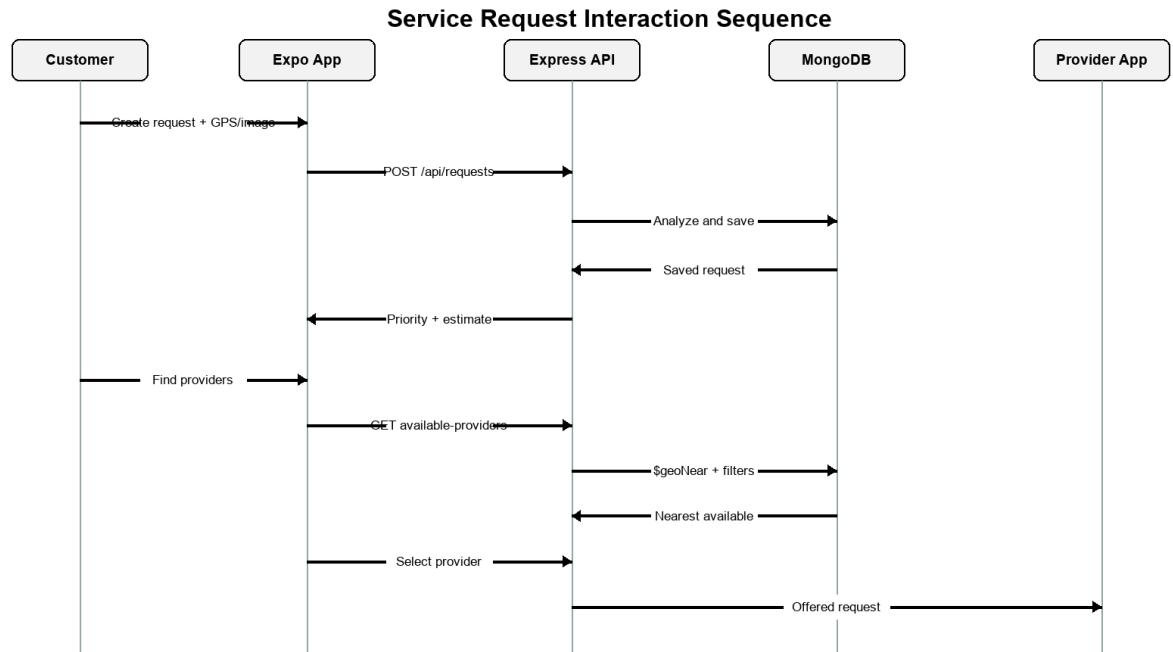


Figure 3-4. Service-request interaction sequence diagram.

3.6 Activity Diagrams

The activity diagram in [Figure 3-5](#) emphasizes state validity. Pending and rejected requests with no provider may enter provider selection. A selected request becomes offered. Rejection clears the provider, preserves rejection history, and lets the customer search again. Acceptance begins the one-way operational sequence. The controller rejects skipped, reversed, repeated, completed, cancelled, or foreign-provider transitions by requiring the expected current status in the database update condition.

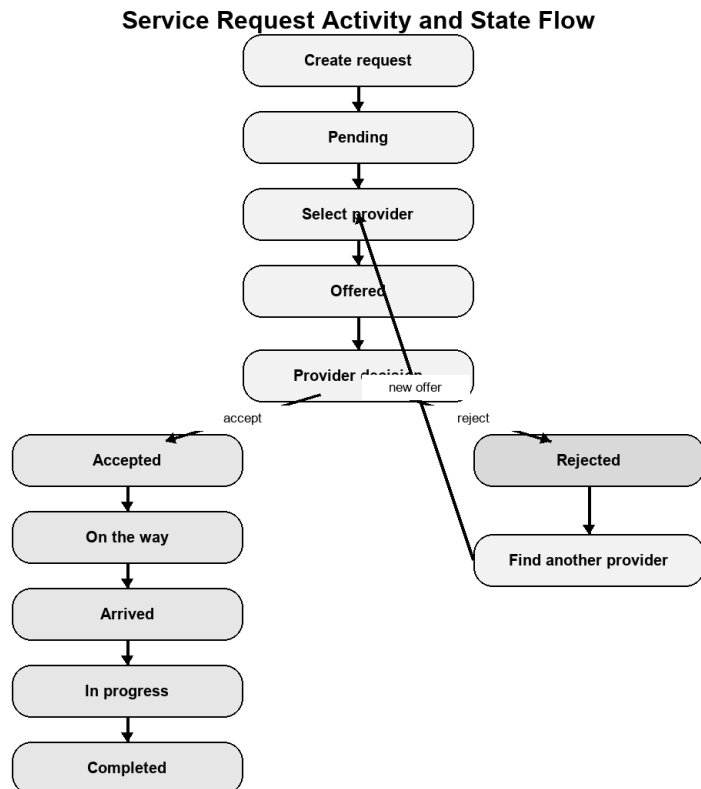


Figure 3-5. Service-request lifecycle activity diagram.

3.7 Entity-Relationship Diagrams

Figure 3-6 presents the logical relationships among the three MongoDB collections. A User may create many ServiceRequests as a customer and may be assigned many as a provider. Each completed request can have zero or one Rating because the rating request reference is unique. Rating redundantly stores the customer and provider references to support ownership validation and provider aggregation. Geospatial point fields on User and ServiceRequest have 2dsphere indexes.

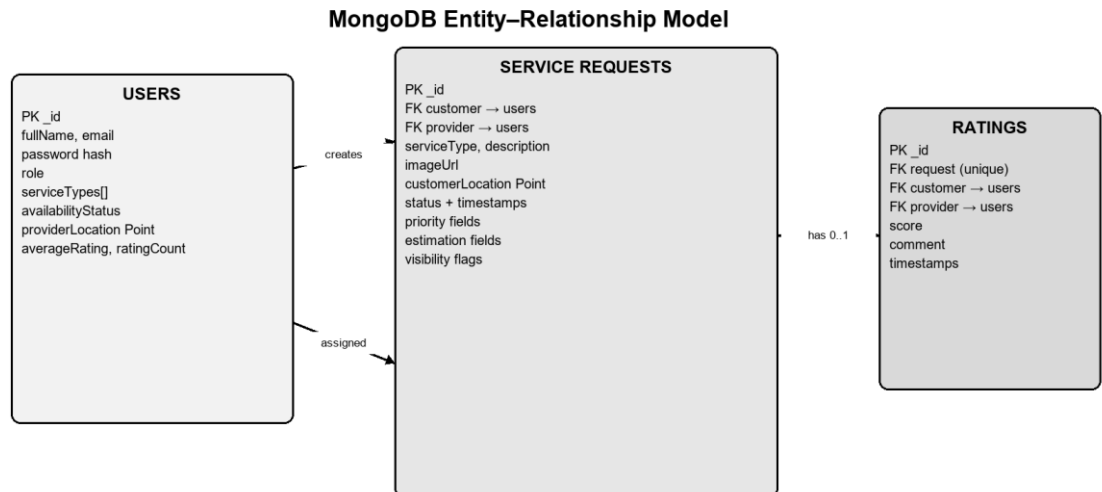


Figure 3-6. MongoDB entity-relationship diagram.

3.8 Non-Technical Aspects

3.8.1 Financial Viability

The current system is an academic prototype, so the analysis distinguishes cash expenditure from student development effort. Open-source frameworks, Expo development tooling, a MongoDB Atlas free tier, and local image storage can support demonstration without license fees. [Table 3-5](#) uses conservative planning assumptions rather than invoices or guaranteed production costs. A real deployment would require revised estimates for hosting, managed storage, domain, monitoring, support, verification, insurance, and legal compliance.

Table 3-5. Prototype financial assumptions.

Cost item	Prototype	
	assumption (USD)	Basis
Software frameworks and development tools	0	Open-source/community tooling
MongoDB Atlas prototype database	0	Free-tier academic prototype assumption
Local upload storage	0	Development computer only
Connectivity and electricity	150	Planning allowance across development
Contingency	100	Unexpected prototype expenses
Total estimated cash cost	250	Excludes student labor and production operation

Prototype Cost–Benefit Overview

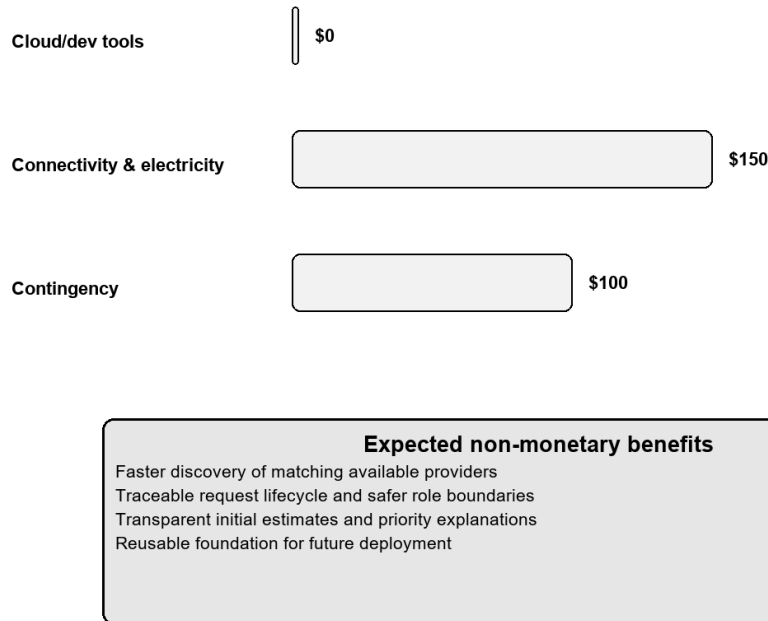


Figure 3-7. Prototype cost–benefit overview.

3.8.2 Stakeholders

Table 3-4. Principal stakeholders.

Stakeholder	Interest/benefit	Potential concern
Customers	Faster access to relevant providers and visible progress	Location/privacy, estimate accuracy, service quality
Service providers	Relevant nearby requests and reputation history	Fair visibility, safety, workload, rejection history
Project team	Academic learning and maintainable system design	Scope, testing, security responsibilities
University/supervisor or	Evidence of engineering analysis and implementation	Accuracy, ethics, originality, documentation
Future operator	Platform continuity and support	Compliance, hosting cost, moderation, incident response

Customers and providers should both have a voice in future usability and safety decisions. Provider visibility rules should be transparent, while customers should understand why location and request information are collected. Any later commercial operator would need formal complaint, verification, moderation, and incident-response processes.

3.8.3 Scope

The implemented scope includes registration, login, session restoration, role navigation, provider services and availability, one-time customer/provider GPS capture, request creation with one optional local image, geospatial discovery, customer selection, provider acceptance/rejection, lifecycle control, automatic five-second refresh, ratings, explainable priority, explainable initial price/duration, and per-user historical cleanup.

The scope explicitly excludes Google Maps, route or traffic ETA, continuous tracking, Socket.IO, push notifications, chat, payment, Cloudinary, Firebase, OpenAI API calls, automatic provider selection, automatic charging, professional licensing verification, and production deployment operations. These items must not be interpreted as completed components.

3.8.4 Risks

Table 3-6. Principal project risks and mitigations.

Risk	Effect	Current/future mitigation
GPS Incorrect or denied	No useful nearby result	Clear permission errors; recapture; server validation
Stale availability	Unavailable provider displayed	Profile reload on focus; acceptance marks busy
Network interruption	Lists cannot refresh	Keep existing data; non-blocking message; retry polling
Concurrent actions	Conflicting provider/status updates	Atomic expected-state database filters
Rule false positive/negative	Misleading priority or estimate	Explain rules; bounds; professional confirmation
Local upload growth	Disk exhaustion/loss	5 MB limit; migrate to managed storage in production
Credential/token compromise	Unauthorized access	bcrypt, JWT verification, SecureStore, HTTPS future requirement
Privacy exposure	Personal/location harm	Minimize returned fields; no exact provider location in discovery

3.8.5 Schedule and Milestones

Table 3-7. Schedule and milestone summary.

Period	Milestone	Principal output
Sep–Oct 2025	Requirements and interface foundation	Scope, navigation, guest/auth screens

Period	Milestone	Principal output
Nov–Dec 2025	Backend and database foundation	Express, authentication, MongoDB
Jan–Feb 2026	Provider and request location	Profiles, GPS, GeoJSON requests
Feb–Mar 2026	Discovery and provider workflow	Nearby filtering, selection, accept/reject
Mar–Apr 2026	Lifecycle and feedback	Status progression, polling, ratings
Apr–May 2026	Media and explainable analysis	Images, priority, price/duration estimate
May–Jun 2026	Verification and documentation	Automated tests, report, demonstration preparation

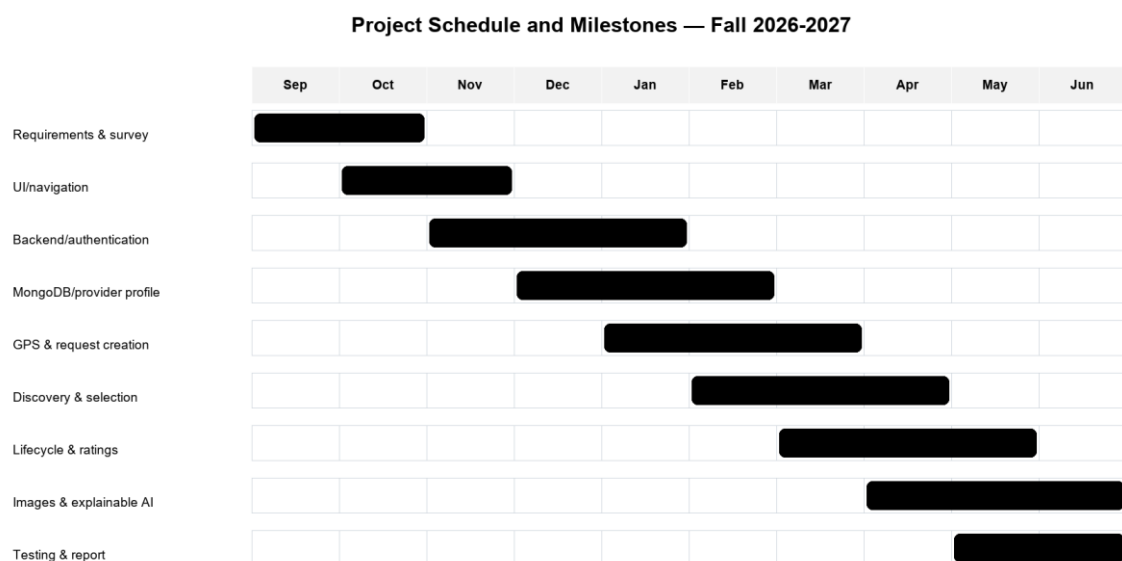


Figure 3-8. Project schedule and milestones.

3.8.6 Ethical and Social Considerations

The most important ethical requirement is that analysis supports rather than replaces human judgment. A keyword match may assist triage, but it cannot confirm electrical safety, diagnose a pipe, determine labor difficulty, or establish a binding price. For that reason, Smart Service presents a reason and a final-price warning, preserves customer provider choice, and requires provider inspection. Future evaluation should measure false classifications in both English and Arabic and involve qualified service professionals.

Location and customer descriptions can reveal sensitive household information. The system therefore requests foreground location only, performs no continuous tracking,

sends only the current coordinate pair, withholds exact provider coordinates before selection, and avoids returning customer email or exact location in offered-request summaries. Production use would require a privacy notice, retention policy, deletion policy, legal review, provider/customer safety processes, and stronger identity verification.

Ratings can improve trust but can also create unfair reputational effects. The current design restricts a rating to the owning customer and one completed request. Future governance should include review moderation, dispute handling, bias monitoring, and safeguards against retaliation or manipulation.

3.8.7 Environmental and Sustainability Considerations

The prototype has limited direct material impact because it is software and reuses existing mobile phones and cloud infrastructure. Digital requests may reduce unnecessary calls or exploratory travel by showing nearby matching providers; however, this benefit has not been quantitatively measured. Cloud computation, mobile networking, stored images, and device charging still consume energy. Sustainable development measures include limiting image size, avoiding continuous GPS, keeping rule analysis computationally small, deleting unnecessary temporary files, and selecting efficient hosting appropriate to actual demand.

3.8.8 Relevant Standards

Table 3-8. Relevant standards and guidance.

Standard/guidance	Relevance to Smart Service	Current application
IETF RFC 7519 — JWT	Compact authenticated claims representation	Seven-day tokens with user ID and role
IETF RFC 7946 — GeoJSON	Interoperable geographic geometry	Point coordinates stored longitude, latitude
MongoDB 2dsphere guidance	Spherical proximity and indexing	Provider/request indexes and \$geoNear
OWASP Password Storage	Adaptive salted password hashing	bcryptjs work factor 12
W3C WCAG 2.2	Perceivable, operable, understandable, robust UI	Accessible roles/status labels partly applied; full audit future
HTTP/REST conventions	Method, status, resource- oriented API behavior	GET, POST, PATCH and safe status codes

Compliance is not claimed solely because a technology or guideline is referenced. The table identifies design relevance and current alignment. Formal conformance testing, penetration testing, accessibility evaluation, privacy impact assessment, and production operational controls remain future work.

3.9 Conclusion

The design combines role-specific Expo navigation, a protected Express API, MongoDB geospatial persistence, deterministic analysis, and an explicit service lifecycle. Ownership and expected-state conditions protect shared requests, while safe response formatting limits unnecessary personal data. The diagrams and tables demonstrate that each major interface action has a corresponding server responsibility and stored-domain representation. The design remains intentionally bounded: it supports an academic prototype and future extension without presenting maps, real-time sockets, payment, or generative AI as completed features.

REFERENCES

- [1] Taskrabbit Support, “How Do I Hire a Tasker?” [Online]. Available: <https://support.taskrabbit.com/hc/en-us/articles/46260422073755-How-Do-I-Hire-a-Tasker>. Accessed: Aug. 2, 2026.
- [2] Angi, “Frequently Asked Questions.” [Online]. Available: <https://www.angi.com/faq/>. Accessed: Aug. 2, 2026.
- [3] Meta Platforms, “React Native: Introduction.” [Online]. Available: <https://reactnative.dev/docs/getting-started>. Accessed: Aug. 2, 2026.
- [4] Expo, “Location — Expo SDK 57 Documentation.” [Online]. Available: <https://docs.expo.dev/versions/v57.0.0/sdk/location/>. Accessed: Aug. 2, 2026.
- [5] H. Butler et al., “The GeoJSON Format,” RFC 7946, Aug. 2016. [Online]. Available: <https://www.rfc-editor.org/info/rfc7946/>. Accessed: Aug. 2, 2026.
- [6] MongoDB, “Geospatial Queries and 2dsphere Indexes.” [Online]. Available: <https://www.mongodb.com/docs/v8.2/geospatial-queries/>. Accessed: Aug. 2, 2026.
- [7] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” RFC 7519, May 2015. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc7519>. Accessed: Aug. 2, 2026.
- [8] OWASP Foundation, “Password Storage Cheat Sheet.” [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html. Accessed: Aug. 2, 2026.
- [9] Taskrabbit, “How It Works.” [Online]. Available: <https://www.taskrabbit.com/>. Accessed: Aug. 2, 2026.
- [10] Taskrabbit Support, “How Do I Become a Tasker?” [Online]. Available: <https://support.taskrabbit.com/hc/en-us/articles/46260467885979-How-Do-I-Become-a-Tasker>. Accessed: Aug. 2, 2026.
- [11] Taskrabbit, “How Pricing Guidance Works on Taskrabbit,” Apr. 13, 2022. [Online]. Available: <https://www.taskrabbit.com/blog/how-pricing-guidance-works-on-taskrabbit/>. Accessed: Aug. 2, 2026.
- [12] Angi, “What’s the Difference Between Opportunities and Leads?” [Online]. Available: <https://intercom.help/angi/en/articles/6229338-what-s-the-difference-between-opportunities-and-leads>. Accessed: Aug. 2, 2026.
- [13] Urban Company, “Service Professional Enablement.” [Online]. Available: <https://investorrelations.urbancompany.com/announcements-and-highlights/service-professional-enablement>. Accessed: Aug. 2, 2026.
- [14] W3C, “Web Content Accessibility Guidelines (WCAG) 2.2,” Dec. 12, 2024. [Online]. Available: <https://www.w3.org/TR/WCAG22/>. Accessed: Aug. 2, 2026.
- [15] Express.js, “Basic Routing.” [Online]. Available: <https://expressjs.com/en/starter/basic-routing/>. Accessed: Aug. 2, 2026.